

A Systematic Instruction-Level Testing Framework for Characterizing Undocumented Behaviors on ARMv8-A Processors

xinzhe Yang^{1,2*}, Second Author^{2,3†} and Third Author^{1,2†}

^{1*}College of Electronics and Information, Southwest Minzu University,
Street, Chengdu, 610225, Sichuan, China.

²Department, Organization, Street, City, 10587, State, Country.

³Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): jonny.y0827@gmail.com;
Contributing authors: iiauthor@gmail.com; iiiauthor@gmail.com;
[†]These authors contributed equally to this work.

Abstract

Ensuring consistency between hardware implementation and architectural specification is a critical challenge in post-silicon verification. This paper presents a systematic testing framework to characterize the functional behavior of undocumented instructions on ARMv8-A processors. Unlike random fuzzing, the proposed framework employs a deterministic multi-dimensional observation strategy. It integrates differential analysis of general-purpose registers to capture state corruption and leverages Performance Monitoring Units (PMUs) to detect implicit memory accesses. Furthermore, to rigorously verify control-flow integrity, we introduce a sentinel-guarded memory layout combined with trap-based fall-through verification. By analyzing signal contexts (SIGTRAP, SIGSEGV, SIGVTALRM), the framework precisely quantifies program counter deviations and execution hangs. Implemented on the RK3399 platform, the method exhaustively evaluated over 1.2 billion architecturally undefined encodings. Experimental results demonstrate the framework's capability to categorize these opcodes into distinct functional classes, offering a practical methodology for enhancing ISA transparency and functional consistency checking.

Keywords: ARMv8-A, Instruction-level testing, Functional characterization, Undocumented opcodes, Post-silicon validation

1 Introduction

The continuous evolution of Modern RISC architectures, exemplified by ARMv8-A [1], has led to increasingly complex instruction set architectures (ISAs) designed to satisfy diverse computing demands. In the lifecycle of processor development, ensuring the consistency between the hardware implementation and the architectural specification is a paramount objective of post-silicon validation. While official documentation explicitly defines the behavior of supported instructions, the vast encoding space outside these definitions—commonly referred to as the *undefined encoding space*—remains a critical blind spot.

Recent studies [3, 8] have demonstrated that commercial processors often execute **undocumented instructions** instead of triggering the expected undefined instruction exceptions. These discrepancies may originate from legacy backward compatibility, incomplete feature disabling, or silicon errata. From a testing and reliability perspective, these **undocumented behaviors** represent non-compliant deviations from the ISA specification. If left uncharacterized, they pose risks to system determinism, potentially leading to silent data corruption or unexpected control flow behaviors in safety-critical applications.

However, existing methodologies for analyzing undefined encodings suffer from significant limitations in observability. Conventional approaches, such as random instruction fuzzing [6] or simple existence scanning [8], primarily focus on the binary question of “*Can this instruction execute?*” rather than the functional question of “*What implies side-effects does it produce?*”. These tools often lack the instrumentation required to capture fine-grained architectural state changes, such as silent register modifications or implicit memory accesses that do not trigger faults. Furthermore, analyzing control-flow behavior without a robust isolation mechanism often leads to nondeterministic system hangs, hindering exhaustive testing.

To address these challenges, this paper proposes a systematic, instruction-level testing framework specifically designed for the functional characterization of undocumented opcodes on ARMv8-A processors. Unlike security-oriented scanners that search for vulnerabilities, our approach adopts a rigorous structural testing methodology. We introduce a multi-dimensional observation strategy that serves as a deterministic test oracle:

1. **Register State Verification:** We employ differential analysis of General-Purpose Registers (GPRs) to detect unarchitected state transitions.
2. **Memory Behavior Monitoring:** We leverage hardware Performance Monitoring Units (PMUs) to capture micro-architectural memory events (Load/Store Retired) that are invisible to standard software debuggers.
3. **Robust Control-Flow Analysis:** We design a novel execution harness using a *sentinel-guarded memory layout* combined with *trap-based fall-through verification*. This mechanism allows precise quantification of Program Counter (PC) deviations and distinguishes between valid branches, illegal jumps, and execution timeouts.

Implemented on the RK3399 platform (Cortex-A53), the proposed framework achieved an exhaustive evaluation of the entire 32-bit undefined encoding space, totaling over 1.2 billion opcodes. The experimental results demonstrate that the framework

can effectively filter out NOP-like placeholders and categorize active undocumented instructions into distinct functional classes based on their behavioral signatures.

The main contributions of this paper are as follows:

- We propose a systematic testing framework for undocumented instructions that integrates register differential analysis, PMU-based memory monitoring, and sentinel-based control flow verification.
- We present an exhaustive characterization of the ARMv8-A AArch32 undefined encoding space, identifying and categorizing millions of executable undocumented opcodes.
- We provide a detailed analysis of non-compliant behaviors, including implicit memory operations and register corruptions, offering valuable insights for ISA transparency and processor verification.

The remainder of this paper is organized as follows. Section 2 reviews related work in processor fuzzing and hidden instruction analysis. Section 3 details the proposed testing methodology and observation mechanisms. Section 4 describes the implementation on the test platform. Section 5 presents the experimental results and behavioral classification. Finally, Section 7 concludes the paper.

2 Related Work

Research on validating undefined instruction behaviors has evolved from simple opcode enumeration to complex consistency testing. This section categorizes prior arts into instruction space exploration, differential consistency testing, and micro-architectural observability.

2.1 Instruction Space Exploration

The primary step in analyzing undocumented behaviors is establishing the boundary between valid and invalid encoding spaces. Domas [3] introduced *Sandsifter*, a depth-first search approach to systematically enumerate the x86 instruction space, revealing a significant discrepancy between hardware implementation and documentation. Following this methodology, Strupe and Kumar [8] proposed *Armshaker* for the ARMv8-A architecture, which scans for executable instructions within the undefined encoding space. Subsequent works like *UISFuzz* [6] and *InsFinder* [4] optimized the scanning efficiency using heuristic algorithms and early-termination strategies. **Limitation:** While these tools effectively identify the *existence* of executable undocumented opcodes (i.e., whether they trigger an exception), they generally treat the instruction execution as a black box. They do not systematically characterize the functional side-effects, such as register state modifications or control flow anomalies, which are critical for reliability analysis.

2.2 Differential Consistency Testing

To understand the semantics of undefined instructions, researchers have adopted differential testing, where hardware execution is compared against a software reference

model (Golden Model). Jiang et al. [5] proposed *Examiner*, which detects inconsistent behaviors between real ARM devices and QEMU emulators. Similarly, Wang et al. [9] and Dofferhoff et al. [2] developed consistency testing models to reveal discrepancies in RISC processors. **Limitation:** These approaches rely heavily on the fidelity of the reference model (e.g., QEMU or Gem5). However, reference models often abstract away micro-architectural details or fail to implement undefined behaviors present in silicon. Consequently, relying solely on model comparison may miss silicon-specific behaviors or yield false positives due to model inaccuracies. Our framework addresses this by performing *self-contained* characterization on the target hardware without requiring an external reference model.

2.3 Micro-architectural Observability

Beyond architectural state (registers and memory), observing micro-architectural events offers deeper insights into instruction behavior. Spisak [7] and Yang et al. [12] demonstrated the utility of Performance Monitoring Units (PMUs) in capturing fine-grained hardware events, such as cache hits and branch predictions. In the x86 domain, Woodruff et al. [10] and Wu et al. [11] utilized side-channel information and performance counters to infer the internal operations of undocumented instructions. **Limitation:** Prior works predominantly utilized PMU and side-channel data for security analysis (e.g., detecting malware or covert channels). Few studies have integrated these micro-architectural observations into a structured functional testing framework. Specifically, there is a lack of unified methodology that simultaneously correlates register state changes, implicit memory traffic (via PMU), and control-flow integrity to construct a complete behavioral profile of undefined opcodes.

In summary, our work bridges these gaps by proposing a holistic testing framework that combines exhaustive enumeration with multi-dimensional instrumentation (Register, PMU, and Control-flow), enabling precise functional characterization without reliance on external reference models.

3 Methodology

We propose a systematic, automated testing framework to characterize the behaviors of undocumented instructions. As illustrated in Fig. 1, the workflow is structured into three distinct phases: (1) Test Corpus Generation, (2) Automated Screening Harness, and (3) Multi-dimensional Functional Characterization. This design ensures that the analysis is exhaustive, reproducible, and fault-tolerant.

3.1 Test Corpus Generation

The first phase involves defining the input domain. We iterate through the entire 32-bit ARM opcode space (2^{32} combinations). To filter out architecturally defined instructions, we employ a specification-accurate disassembler derived from the ARM machine-readable specifications (XML). This process effectively reduces the search space from 4.29 billion to approximately 1.2 billion candidates, forming the *Candidate Undocumented Corpus*.

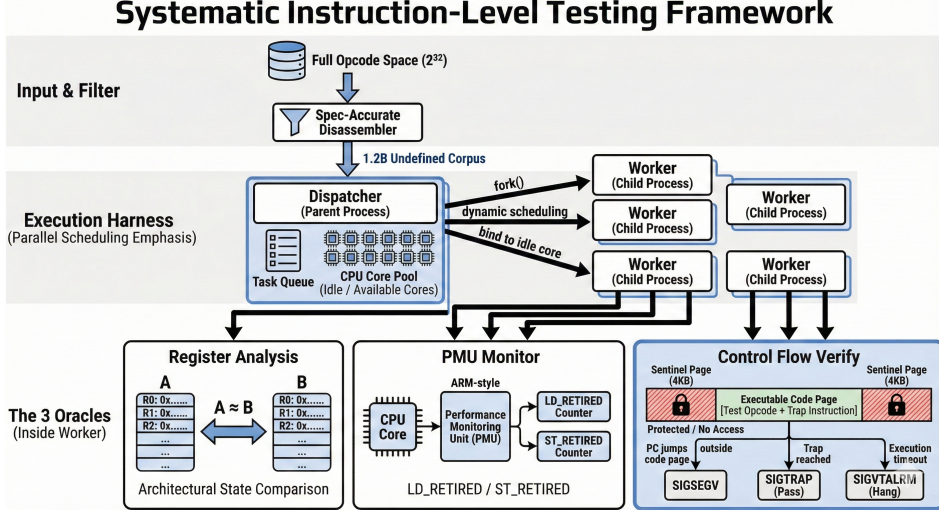


Fig. 1 The overall workflow of the proposed systematic instruction-level testing framework. It illustrates the stages from test corpus generation and parallel dispatching to multi-dimensional functional characterization (Register, PMU, and Control Flow). The callout box details the sentinel-guarded memory layout used for deterministic control-flow verification.

Clarification on Reference Dependency: It is important to note that this disassembler acts as a **Static Decoding Oracle**. While our framework relies on this oracle to identify *what* is undefined, the subsequent phase—characterizing *how* these undefined instructions behave in silicon—is entirely **reference-free**. We do not rely on emulation models (e.g., QEMU) for behavioral prediction, as they often lack fidelity for undocumented cases. This distinction ensures that our corpus is compliant with the standard, while our behavioral results capture the ground-truth silicon implementation.

3.2 Automated Screening Harness

Executing undefined instructions inherently risks crashing the processor or corrupting the OS kernel. To mitigate this, we designed a robust *Sandboxed Dispatcher-Worker* architecture:

- **Process Isolation:** The framework operates in a parent-child mode. The parent process acts as a **Dispatcher**, managing the instruction corpus and logging results. The child process acts as a **Worker**, executing the target **undocumented opcode**.
- **Parallel Execution:** The dispatcher distributes workload across available cores (e.g., A53 cores on RK3399) to maximize throughput.
- **Fault Tolerance:** If a worker crashes (e.g., due to an illegal instruction exception), the operating system signal handler captures the event, and the parent process spawns a new worker to continue the sequence.

This screening phase filters the 1.2 billion candidates down to a subset of *executable undocumented instructions* (i.e., those that do not trigger an immediate Undefined Instruction Exception), which are then subjected to deep functional characterization.

3.3 Multi-dimensional Functional Characterization

To comprehensively describe the behavior of the screened instructions, we implement three independent *Test Oracles*.

3.3.1 Register State Differential Analysis

To detect explicit or implicit data manipulations, we capture the architectural state—including all General-Purpose Registers (R0-R12), the Link Register (LR), and the Stack Pointer (SP)—immediately before and after the execution of the target instruction. By computing the bitwise difference between the pre- and post-execution snapshots, the framework identifies specific registers corrupted by the undocumented opcode.

3.3.2 Implicit Memory Access Monitoring

Some undocumented instructions may trigger memory operations without altering register states. We leverage the hardware Performance Monitoring Unit (PMU) to detect these invisible side effects. Specifically, we configure the PMU to count `LD_RETIRE` (load operations) and `ST_RETIRE` (store operations) events. An increment in these counters during the isolated execution of a single target instruction serves as a definitive indicator of implicit memory traffic.

3.3.3 Deterministic Control-Flow Verification

A critical challenge in testing undefined opcodes is characterizing their impact on the Program Counter (PC). We introduce a **Sentinel-Guarded, Trap-Based Verification** mechanism to quantify PC deviations precisely:

1. **Sentinel Memory Layout:** As shown in the memory map design, we allocate a dedicated executable page for the target instruction. We then map two 4KB *Sentinel Pages* immediately before and after this executable page.¹ These sentinels serve as catch-basins for erratic jumps.
2. **Trap Injection:** Inside the executable page, we inject a `TRAP` instruction (e.g., a specific breakpoint) immediately following the target undocumented opcode. This creates a deterministic "fall-through" expectation.
3. **Signal-Based Behavior Inference:** We register custom signal handlers to intercept execution outcomes, classifying the control flow behavior based on the received signal and context (`'uc_mcontext'`):
 - **Normal Fall-through (SIGTRAP):** If the system reports a `SIGTRAP` originating from the injected trap instruction, we conclude the target opcode did not alter the control flow.
 - **PC Deviation (SIGSEGV):** If the PC jumps to one of the Sentinel Pages, a `SIGSEGV` is triggered. The handler extracts the faulting address from

¹The 4KB range is sufficient because most accidental branches in 32-bit ARM immediate encodings have limited relative offsets. Long jumps typically require register-based addressing, which would necessitate valid GPR values, inherently reducing the probability of jumping blindly beyond the sentinel.

`'uc_mcontext.arm_pc'`, allowing us to calculate the exact jump offset (in bytes) relative to the instruction address.

- **Execution Hang (SIGVTALRM):** A software watchdog timer is armed before execution. If the instruction causes a pipeline stall or an infinite loop (self-spin), the watchdog triggers SIGVTALRM, identifying the instruction as a "Hang" type.

This methodology ensures that even complex control-flow anomalies are not just detected as "crashes" but are quantitatively measured.

4 Implementation

This section details the concrete realization of the proposed framework on a commercial ARMv8-A platform, highlighting the engineering choices made to guarantee measurement fidelity and reproducibility.

4.1 Platform Configuration and Isolation

The experimental evaluation was conducted on a Rockchip RK3399 SoC, which features a heterogeneous big.LITTLE architecture. To minimize micro-architectural noise (e.g., cache contention and context switching) and ensure the determinism of timing measurements, we enforced strict hardware isolation:

- **Core Affinity:** The testing workload was pinned exclusively to a single Cortex-A53 core (1.4 GHz) using `sched_setaffinity`. The remaining cores were isolated to handle OS background tasks.
- **OS Environment:** The system runs Ubuntu 20.04 (Linux kernel 5.18) with full access to the Performance Monitoring Unit (PMU) enabled via a custom kernel module.
- **Execution State:** All experiments were conducted in the AArch32 execution state, as the 32-bit opcode space allows for exhaustive enumeration compared to the sparse 64-bit space.
- **Execution State Selection (AArch32 vs. AArch64):** Although AArch64 is the primary execution mode for modern workloads, our preliminary investigation revealed a distinct dichotomy in hardware behavior. We applied the proposed screening harness to the undocumented encoding space of the AArch64 instruction set. The results were conclusive: **zero** undocumented instructions were executable. Every non-architectural encoding in AArch64 triggered an immediate undefined instruction exception (SIGILL).

This finding indicates that the hardware implementation of the newer AArch64 state strictly adheres to the architectural specification. In contrast, the legacy AArch32 state, constrained by backward compatibility and complex decoding logic, exhibits significant implementation divergence. Consequently, this paper focuses exclusively on the AArch32 execution state, where the risk of undocumented behaviors is concentrated.

4.2 Test Harness Implementation

The harness is implemented as a multi-process application to support the Dispatcher-Worker architecture described in Section 3. **Instruction Injection:** Each candidate opcode is injected into a dynamically allocated executable page. To preserve the purity of the register state, the injection harness is written in naked inline assembly. This ensures that the framework’s own stack usage does not pollute the register snapshots of the instruction under test. **Data Sharing:** Communication between the parent (Dispatcher) and child (Worker) processes is optimized using POSIX shared memory (`shm_open`). This low-latency channel is used to transmit the test opcode and retrieve the resulting register context and PMU counters without the overhead of pipe serialization.

4.3 Realization of Test Oracles

4.3.1 Register Context Capture

The Register Oracle requires cycle-accurate snapshots. We implemented a custom context switch routine that: 1. Pushes the current context (R0-R12, LR, SP, PC) to a dedicated stack in shared memory. 2. Executes the target undocumented opcode. 3. Immediately saves the post-execution context. This implementation allows the framework to detect even single-bit flips in any General-Purpose Register.

4.3.2 Zero-Overhead PMU Event Monitoring

Standard OS-level profiling tools (e.g., `perf`) often introduce non-deterministic overhead due to system calls and context switching. To eliminate this kernel-induced noise and achieve cycle-accurate fidelity, we enabled direct user-space access to the PMU hardware by setting the User Enable Register (`PMUSERENR_ELO`) via a lightweight kernel module.

Consequently, our test harness configures, enables, and reads the PMU counters (specifically `LD_RETIRED` and `ST_RETIRED`) directly using **naked inline assembly**. We focused on these architectural retirement events because they represent the most critical side-effect: **Memory State Change**. While micro-architectural events like cache hits/misses or TLB accesses could imply activity, they are often noisy and implementation-dependent. Retirement counters provide a deterministic confirmation that the instruction functionally interacted with the memory hierarchy. This approach completely bypasses the operating system stack during the critical execution window. By isolating the measurement from kernel background noise, the captured event counts are attributed exclusively to the target instruction, rendering the result theoretically noise-free. As a validation step, we continue to employ the NOP-reference baseline to confirm the calibration before every test batch.

4.3.3 Sentinel-Guarded Memory Layout

To implement the deterministic control-flow verification described in Section 3.3.3, we utilized the Linux `mmap` system call to construct a precise virtual memory layout:

- **The Trap Guard:** Within the executable page, a valid trap instruction (opcode 0xE7FFDEFE for AArch32) is hard-coded immediately following the test opcode slot.
- **The Sentinel Pages:** Two 4KB pages are mapped immediately preceding and succeeding the executable code page. These pages are protected with `PROT_NONE`, ensuring that any PC jump landing in these regions triggers an immediate memory access violation (SIGSEGV).

Signal Handling Logic: A custom `sigaction` handler is registered to intercept SIGTRAP, SIGSEGV, and SIGVTALRM. As shown in Algorithm 1, the handler inspects the `uc_mcontext` structure to recover the faulting PC address, calculating the precise jump offset relative to the instruction address.

Algorithm 1 Signal-Based Control Flow Classification

Require: Signal S , Context ctx , OpcodeAddress $Addr_{op}$

Ensure: Behavior Class C , Offset Δ

```

1:  $C \leftarrow \text{UNKNOWN}$ 
2:  $\Delta \leftarrow 0$ 
3: if  $S = \text{SIGTRAP}$  then
4:    $C \leftarrow \text{NORMAL\_FALLTHROUGH}$ 
5:    $\Delta \leftarrow 4$  ▷ Standard execution step
6: else if  $S = \text{SIGVTALRM}$  then
7:    $C \leftarrow \text{HANG\_TIMEOUT}$ 
8: else if  $S = \text{SIGSEGV}$  then
9:    $Addr_{fault} \leftarrow ctx \rightarrow \text{arm\_pc}$  ▷ Extract fault address from context
10:  if  $Addr_{fault} \in \text{Sentinel\_Pages}$  then
11:     $C \leftarrow \text{PC\_DEVIATION}$ 
12:     $\Delta \leftarrow Addr_{fault} - Addr_{op}$  ▷ Calculate jump offset
13:  else
14:     $C \leftarrow \text{CRASH\_MEM\_VIOLATION}$  ▷ Wild access outside sentinels
15:  end if
16: end if
17: return  $(C, \Delta)$ 

```

5 Experimental Results

This section presents the quantitative findings of our exhaustive testing campaign on the RK3399 platform. We analyze the instruction throughput, the yield of executable undocumented opcodes, and their functional categorization based on the proposed multi-dimensional test oracles.

5.1 Screening Efficiency and Coverage

The testing campaign targeted the entire 32-bit undefined encoding space, consisting of $N_{total} = 1,203,581,952$ candidates. Leveraging the parallel Dispatcher-Worker

architecture on 4 Cortex-A53 cores, the full exhaustive scan was completed in approximately 7.4 hours. The average throughput was calculated at $\approx 11,300$ instructions per second per core, totaling $\approx 45,200$ instructions per second system-wide.

To verify the reproducibility of our results, the screening phase was repeated four times. As shown in the logs, three of the four runs produced highly consistent counts of executable instructions (deviating by less than 5 opcodes), demonstrating the stability of the test harness. We adopt the maximum stable yield for detailed analysis:

- **Total Undefined Candidates:** 1,203,581,952
- **Executable Undocumented Opcodes:** 105,738,315 (Yield $\approx 8.79\%$)

These 105.7 million opcodes successfully executed without triggering an immediate Undefined Instruction Exception (SIGILL) and were subsequently passed to the functional characterization stage.

5.2 Functional Characterization

Table 1 summarizes the classification of the 105.7 million executable undocumented opcodes based on their observable side effects.

Table 1 Functional Characterization of Undocumented Opcodes on Cortex-A53

Category	Observed Behavior	Count	Prevalence
GPR Modifying	Modifies R0-R12 (Silent Data Corruption)	13,498,268	12.77%
Flag Modifying	Updates CPSR flags (N, Z, C, V)	1,921,032	1.82%
SP Corrupting	Corrupts Stack Pointer (Context Loss)	21,896	0.02%
Memory Load	Implicit Load (PMU LD_RETIRED > 0)	3,186,712	3.01%
Memory Store	Implicit Store (PMU ST_RETIRED > 0)	4,184	< 0.01%
Control Flow	PC Deviation or System Hang	0	0.00%

5.2.1 Register and State Anomalies

A significant portion (12.77%) of the undocumented opcodes silently modified General-Purpose Registers (GPRs). Furthermore, 1.92 million instructions altered the condition flags (N, Z, C, V) in the Current Program Status Register (CPSR). Crucially, our testing confirmed that no undocumented instruction succeeded in modifying the privileged bits of the CPSR (e.g., Mode bits, IRQ/FIQ masks), indicating that the hardware privilege isolation remains robust against these undefined behaviors.

5.2.2 Stack Integrity Violations

We identified 21,896 instructions that caused **Stack Pointer (SP) Corruption**. In our test harness, this behavior was detected indirectly: the execution of these instructions corrupted the SP, causing the subsequent context restoration routine (which

relies on the stack) to pop invalid data ("garbage values" or zeros) back into the registers. This highlights a critical reliability risk, as such instructions can silently crash the application stack frame without generating an immediate fault signal.

5.2.3 Implicit Memory Traffic

Leveraging the zero-overhead PMU monitoring, we detected 3.19 million instructions performing implicit loads and 4,184 performing implicit stores. The validity of these PMU readings was confirmed by cross-referencing with NOP instructions (which consistently registered zero events) and known load/store instructions, ensuring that the detected events represent genuine micro-architectural memory traffic.

5.2.4 Control Flow Integrity

Contrary to findings in other architectures, our sentinel-guarded verification revealed **zero** instances of silent PC deviation or true system hangs. Instructions that appeared to "timeout" during the initial screening were confirmed to be slow-recovering crashes (SIGILL/SIGSEGV) rather than infinite loops. This result suggests that the Cortex-A53 implementation strictly enforces architectural traps for undefined control-flow operations, preventing non-deterministic jumps into the sentinel regions.

5.2.5 Spatial Distribution Analysis

As visualized in Fig. 2, the undocumented instructions are not randomly scattered but exhibit strong *spatial locality* and *periodicity*.

For instance, the GPR-modifying opcodes (Fig. 2e) appear in regular intervals across the entire address space, suggesting they share common decoding logic with defined arithmetic instructions. The zoomed-in view (Fig. 2j) further reveals that these opcodes often form contiguous blocks rather than isolated points.

This clustering characteristic is consistent across other categories as well (e.g., Load instructions in Fig. 2b and g). These patterns validate our hypothesis that undocumented behaviors stem from incomplete decoding logic in specific functional units (such as the arithmetic ALU or Load/Store unit), rather than random silicon defects or bit-flips.

5.3 Root Cause Analysis: Case Studies on Decoding Anomalies

While the previous sections categorized the observable behaviors, it is crucial to understand the micro-architectural mechanisms enabling these executions. Our analysis suggests that these behaviors stem from *Incomplete Decoding Logic* and *Pipeline Hazard Negligence*, where the hardware decoder omits specific bit-checks to optimize area or timing.

We present two representative case studies to demonstrate these decoding gaps.

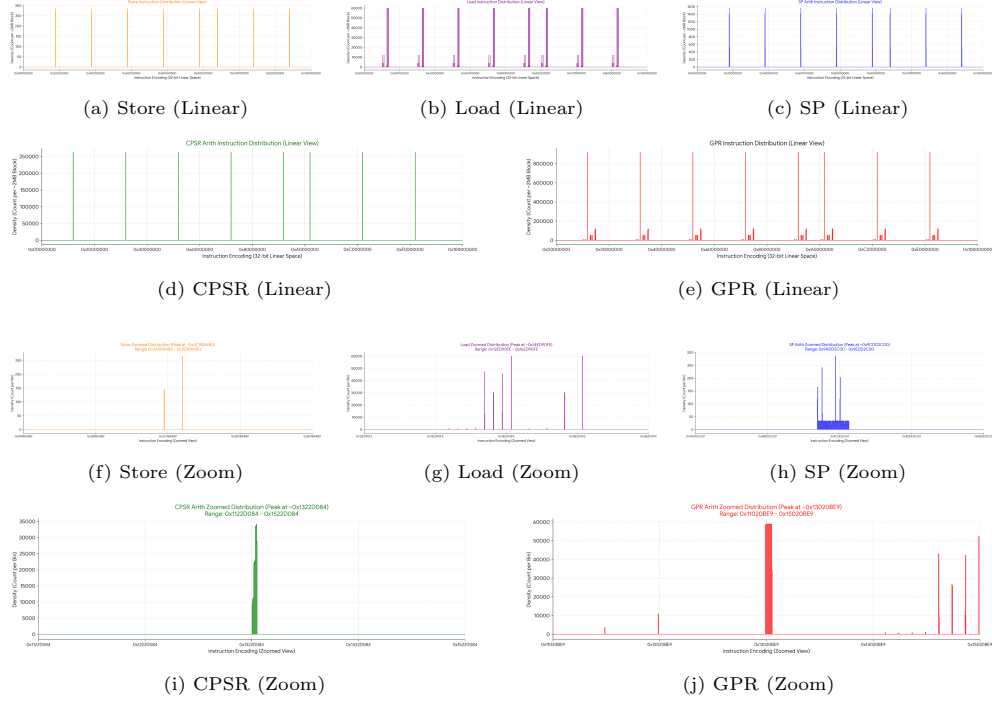


Fig. 2 Comprehensive distribution visualization. Panels (a-e) illustrate the density of undocumented instructions across the full 32-bit linear space, adopting a 3-then-2 layout to maximize visibility. Panels (f-j) provide the corresponding zoomed-in views of the densest regions. Note that GPR and CPSR categories (d, e, i, j) are displayed with enlarged dimensions to highlight their complex clustering patterns.

5.3.1 Case Study A: The "Vacuum State" Alias (TEQ/MSR Conflict)

The opcode `0x332A3116` was identified as an executable instruction that modifies the CPSR flags (specifically setting the N bit), effectively behaving as a comparison operation. However, an architectural analysis reveals it occupies a "definition vacuum" between two valid instructions.

Table 2 details the bitwise conflict. The instruction class is identified by bits 27:25 (001) and Opcode 1001.

- **TEQ Constraint:** The TEQ (Test Equivalence) instruction requires the S-bit (bit 20) to be Set (1) to update flags.
- **MSR Constraint:** The MSR (Move to Status Register) instruction allows the S-bit to be Clear (0) but mandates that the destination field Rd (bits 15:12) be `0b1111` (masking unused bits).

Mechanism Analysis: The target instruction possesses $S = 0$ (violating TEQ) and $Rd \neq 1111$ (violating MSR). Architecturally, this should trigger an Undefined Instruction Exception. However, our experiments confirm that the Cortex-A53 executes it as a TEQ (performing $Rn \oplus Imm$). This indicates that the hardware decoder

Table 2 Bit-level conflict analysis for Opcode 0x332A3116. The instruction violates the constraints of both candidate architectural definitions (TEQ and MSR), yet executes as a TEQ alias on the Cortex-A53.

Instruction Bit Range	Type 27:25	Opcode 24:21	S-Bit 20	Rn 19:16	Rd 15:12	Result Arch Status
0x332A3116	001	1001	0	1010	0011	UNDEFINED
Spec: TEQ	001	1001	1	Rn	(SBZ)	Valid
Spec: MSR	001	1001	0	Mask	1111	Valid

simplifies the logic for Opcode 1001, prioritizing the ALU datapath over the strict validation of the S-bit. It treats the S-bit as a "Don't Care" in this specific encoding context, creating a functional alias.

From a hardware design perspective, this is likely a result of **Area Optimization** or **Logic Minimization**. In the decoding stage, checking every single constraint bit consumes logic gates and increases the critical path delay. Since Opcode 1001 with $S = 0$ is architecturally undefined (unless aliased to MSR), the designers likely optimized the decoder by merging the logic for $S = 0$ and $S = 1$ into a single datapath trigger for the ALU. This suggests that unless a specific "Trap Logic" is explicitly implemented for the $S = 0$ case, the hardware defaults to the most physically proximate valid behavior—in this case, the TEQ operation.

5.3.2 Case Study B: Pipeline Execution of UNPREDICTABLE Loads

The opcode 0x317F20B0 was flagged by our PMU oracle as triggering a memory load event (LD_RETIRED incremented), yet it exhibited unstable behavior. Decoding this instruction reveals a violation of architectural constraints regarding the Program Counter (PC).

As shown in Table 3, the instruction decodes to an LDRH (Load Register Halfword) with Immediate Offset.

- **Base Register (Rn):** Set to 0b1111, which represents the PC.
- **Write-back (W):** Set to 1 (Pre-indexed addressing with write-back).

Table 3 Analysis of the UNPREDICTABLE Load Instruction 0x317F20B0. Attempting to write-back to the PC during a load operation violates architectural rules, yet the instruction successfully passes the decode stage and triggers the Load/Store Unit.

Instruction Bit Range	Class 27:25	P/U/I/W 24:21	L-Bit 20	Rn 19:16	Rt 15:12	Result Arch Status
0x317F20B0	000	1011 (W=1)	1	1111 (PC)	0010	UNPRED
Spec: LDRH	000	1010 (W=0)	1	1111 (PC)	Rt	Valid (Literal)

Mechanism Analysis: The ARMv8 specification explicitly marks LDRH with $Rn = PC$ and Write-back ($W = 1$) as **UNPREDICTABLE**. This combination implies modifying the PC (instruction flow) based on an ALU calculation ($PC - 0$) while simultaneously loading data. Our observation that LD_RETIRE increments suggests that the instruction successfully passes the *Decode* and *Issue* stages. The Load/Store Unit (LSU) receives the μ -op and executes the memory fetch. The "Unpredictability" likely manifests in the write-back stage: the processor may suppress the PC update to prevent pipeline corruption, or the behavior may be indeterminate. This proves that "UNPREDICTABLE" instructions can still activate functional units and consume power/bandwidth, even if their architectural results are voided.

6 Discussion

6.1 Legacy Complexity vs. Modern Simplicity

A pivotal finding of our study is the stark contrast between the AArch32 and AArch64 execution states on the same silicon. While AArch32 exhibited over 100 million executable undocumented opcodes, AArch64 demonstrated strict adherence to the specification with zero executable undefined instructions.

This dichotomy likely stems from the differing design philosophies of the two instruction sets. AArch32 carries the burden of decades of legacy backward compatibility, resulting in a dense and complex variable-length encoding space (especially with Thumb-2) where "encoding holes" are scattered and irregular. Hardware decoders often utilize simplified logic to handle this complexity, inadvertently creating aliases.

In contrast, AArch64 was designed with a "clean slate" approach, featuring a fixed-length, sparse, and regular encoding structure. This allows hardware designers to implement **Default-Fault** decoding logic more efficiently—any bit pattern not explicitly matched in the sparse decoding tree is routed to the illegal instruction exception handler by default. This suggests that modern ISA designs significantly mitigate the risk of undocumented instructions through architectural simplicity.

7 Conclusion

This paper presents a systematic, end-to-end framework for the functional characterization of undocumented instructions on commercial ARMv8-A processors. By integrating a parallelized screening harness with deterministic test oracles—including register differential analysis, zero-overhead PMU monitoring, and sentinel-guarded control-flow verification—we achieved an exhaustive evaluation of the 32-bit undefined encoding space.

The experimental campaign on the RK3399 platform screened over 1.2 billion architecturally undefined encodings in 7.4 hours, achieving a sustained throughput of 11,300 instructions per second per core. Our analysis revealed that approximately 8.79% (105.7 million) of these candidates are executable in hardware. Unlike previous studies that focused primarily on existence scanning, our framework successfully categorized these opcodes into distinct functional classes. We identified significant subsets causing silent data corruption in general-purpose registers (12.77%) and implicit

memory traffic (3.01%). Notably, our precise control-flow verification confirmed that the Cortex-A53 implementation enforces robust exception handling for undefined branches, with zero instances of silent PC deviation detected.

These findings allow us to reframe the understanding of undocumented instructions on this platform. Rather than presenting critical security vulnerabilities (such as control-flow hijacking), the identified behaviors primarily represent **Functional Reliability** concerns. The widespread presence of "silent" instructions (NOP-like or GPR-modifying) indicates loose decoding, but the absence of erratic jumps confirms that the hardware's exception isolation mechanism remains intact and robust.

Future work will focus on three directions: (1) extending the state-space exploration to include varied CPSR flag combinations and privileged execution modes; (2) adapting the test harness to other architectures, such as RISC-V, to perform cross-platform consistency benchmarking; and (3) integrating this testing methodology into continuous integration flows for open-source processor verification.

Declarations

- **Funding:** [Please insert funding details here if any]
- **Conflict of Interest:** The authors declare that they have no conflict of interest.
- **Availability of data and materials:** The instruction corpus and test harness code are available at <https://github.com/Yang-xinzhe/armv8-hidden-inst-detector>.
- **Authors' contributions:** [Author Name] designed the framework and performed the experiments. [Author Name] supervised the project and revised the manuscript. All authors read and approved the final manuscript.

References

- [1] Arm Ltd. *Arm Architecture Reference Manual for A-profile architecture*. Arm Ltd., 2022. Document ID: DDI 0487H.a.
- [2] Rens Dofferhoff, Michael Göebel, Kristian Rietveld, and Erik Van Der Kouwe. iscanu: A portable scanner for undocumented instructions on risc processors. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 306–317. IEEE, 2020.
- [3] Christopher Domas. Breaking the x86 isa. *Black Hat*, 1:1–6, 2017.
- [4] Renhai Dong, Baojiang Cui, Yi Sun, and Jun Yang. Ins finder: A practical cpu undocumented instruction detection framework. *Journal of Circuits, Systems and Computers*, 33(03):2450047, 2024.
- [5] Muhui Jiang, Tianyi Xu, Yajin Zhou, Yufeng Hu, Ming Zhong, Lei Wu, Xiapu Luo, and Kui Ren. Automatically locating arm instructions deviation between real devices and cpu emulators. *arXiv preprint arXiv:2105.14273*, 2021.
- [6] Xixing Li, Zehui Wu, Qiang Wei, and Haolan Wu. Uisfuzz: An efficient fuzzing method for cpu undocumented instruction searching. *IEEE Access*, 7:149224–149236, 2019.
- [7] Matt Spisak. {Hardware-Assisted} rootkits: Abusing performance counters on the {ARM} and x86 architectures. In *10th USENIX Workshop on Offensive Technologies (WOOT 16)*, 2016.

- [8] Fredrik Strupe and Rakesh Kumar. Uncovering hidden instructions in armv8-a implementations. In *Proceedings of the 9th International Workshop on Hardware and Architectural Support for Security and Privacy*, pages 1–9, 2020.
- [9] Yuze Wang, Peng Liu, Weidong Wang, Xiaohang Wang, and Yingtao Jiang. On a consistency testing model and strategy for revealing risc processor’s dark instructions and vulnerabilities. *IEEE Transactions on Computers*, 71(7):1586–1597, 2021.
- [10] William Woodruff, Niki Carroll, and Sebastiaan Peters. Differential analysis of x86-64 instruction decoders. In *2021 IEEE Security and Privacy Workshops (SPW)*, pages 152–161. IEEE, 2021.
- [11] Jiatong Wu, Baojiang Cui, Chen Chen, and Xiang Long. A high efficiency and accuracy method for x86 undocumented instruction detection and classification. In *International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*, pages 295–303. Springer, 2021.
- [12] Yihao Yang, Pengfei Qiu, Chunlu Wang, Yu Jin, Qiang Gao, Xiaoyong Li, DongSheng Wang, and Gang Qu. Exploration and exploitation of hidden pmu events. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2023.